# INTERNSHIP DAILY LOG

## Day 1 – Day 80

*Project: ML-Based Genetic Disease Prediction Web Application*

**Medha U Gowda**
Department of Computer Science and Engineering
Batch of 2026

## Day 1: Introduction to Project Development and Environment Setup

### Nature of Work

On the first day, an introduction to the Machine Learning based project was given. The objective of the project was discussed, which is to design and develop a full stack ML-based web application. The overall architecture of the application was explained, including the integration of frontend, backend, and database components.

The frontend of the application will be developed using web technologies, while the backend will be implemented using the Flask framework in Python. A cloud-based database will be used for storing and managing user and application data. The importance of API testing was also discussed, and Postman was introduced as a tool for testing backend APIs.

Required software tools were identified and installed. PyCharm was selected for frontend development, Spyder IDE for backend development (available through Anaconda distribution), and Postman for testing the application. The development environment was then set up successfully.

A basic backend file ( api.py ) was created in Spyder to initialize the Flask application. The process of running the backend server through the command prompt was observed. Necessary Python libraries such as Flask and Pyrebase were installed using pip to resolve dependency errors.

### Observations and Discussions

The interaction between frontend, backend, and database was discussed in detail. A conceptual flow of the application was explained, helping in understanding how user requests are processed through the system. The importance of proper environment setup for smooth development was observed.

## Day 2: Backend Account Creation and Firebase Integration

### Nature of Work

On the second day, backend development was initiated by writing code to create a simple user account creation module using Flask in api.py . The backend API was designed to accept user details such as user ID, password, and confirm password.

The account creation API was tested using Postman . Different input values were passed to verify conditional checks such as password matching and successful account creation. The response messages helped in confirming whether the API was functioning correctly.

Firebase was introduced as the cloud database for the project. A new Firebase project was created and configured. Authentication and Realtime Database services were enabled, and database rules were modified to allow testing access. The Firebase configuration code was generated and added to a connection.py file.

A proper backend folder structure was maintained by organizing database-related configuration files inside a separate database folder. To improve code maintainability, a properties.py file was created to store success, failure, and error messages as constants.

### Observations and Discussions

The importance of API testing before frontend integration was observed. Proper database connectivity and structured file organization were discussed to ensure scalability and clarity in the project.

## Day 3: Input Validation and Personal Details Module

**Nature of Work**

On the third day, input validation was implemented to ensure correctness of user-entered personal details. A new folder named service was created inside the backend directory to separate validation logic from the main application code.

A file named validation.py was created inside the service folder. This file contains validation functions to check the eligibility of personal details , including first name, last name, email ID, and contact number. The validation checks ensure proper data length and verify whether the inputs contain only valid characters such as alphabets or digits where required.

The validation module was imported into the main backend application file and integrated with the personal details API. After implementing the validation logic, the backend application was executed again.

The personal details API was tested using Postman by providing different input values. Upon successful validation and submission, the entered personal details were stored in the Firebase Realtime Database . The stored data was verified by viewing it in the Firebase console through a web browser.

**Observations and Discussions**

It was observed that separating validation logic into a service module improves code readability and maintainability. Validation helps prevent incorrect data from being stored in the database and ensures data integrity.

## Day 4: Linking Modules and Login with CAPTCHA

**Nature of Work**

On the fourth day, integration between the account creation and personal details modules was completed. The user ID created during account registration was linked with the personal details module. To ensure correct linkage, the same user ID was entered while creating both the account and the personal details.

The backend code was further extended in api.py to implement the login functionality . At this stage, the backend consisted of four main components:

API logic ( api.py )

Database connection ( connection.py )

Validation services ( validation.py )

Constant messages ( properties.py )

Firebase was used for Realtime Database and Authentication to store and verify user credentials.

A CAPTCHA mechanism was implemented in the backend to improve login security. A random number between 0 and 1000 was generated each time the login API was called. During login, the user ID, password, and CAPTCHA value were verified. Login was successful only when both credentials and CAPTCHA matched correctly.

All login operations were tested using Postman by opening new request tabs and sending login data. The responses confirmed whether authentication and CAPTCHA validation were successful.

**Observations and Discussions**

It was observed that linking user ID across modules ensures data consistency, and implementing CAPTCHA adds an additional layer of security to the login process.

## Day 5: Login Validation and Retrieving Personal Details

**Nature of Work**

On the fifth day, the login functionality was completed using User ID, Password, and CAPTCHA . A CAPTCHA was generated in the backend and validated during login testing using Postman . Initially, the login response returned a simple success message. This was later refined using a try–except block to handle errors effectively.

The complete workflow was tested sequentially using Postman:

Create Account

Enter Personal Details

CAPTCHA Generation

Login Verification

If all inputs were correct, a success message was returned; otherwise, appropriate error messages were displayed. This confirmed that the login module was functioning correctly.

After completing login validation, a new backend API was created to retrieve personal details based on User ID ( retrievepd/<userid> ). When an invalid User ID was provided, the system returned a “User ID does not exist” message. When a valid User ID was entered, the API successfully retrieved and displayed the user’s personal details such as first name, last name, contact number, email ID, and user ID.

All API responses were tested using new Postman requests, and previously created Postman collections were saved for future reference. This marked the successful completion of the core backend functionality .

**Observations and Discussions**

It was observed that proper exception handling and step-by-step API testing ensures reliable backend operations and accurate data retrieval.

## Day 6: Frontend Development – Login Page

**Nature of Work**

On the sixth day, frontend development was initiated using Tkinter in Python. A new file named application.py was created and saved inside the frontend folder . Required Tkinter modules were imported, and the main application window was created using mainloop() .

Basic window properties were defined such as:

Application title

Zoomed window state

Window layout

A login frame was created and placed on the main application window. Labels and entry fields were added for User ID , Password , and CAPTCHA , along with a heading label “Login to Application” . When executed, the interface displayed a white background with a blue login frame and text.

To maintain consistent styling, a new folder named properties was created inside the frontend. A file named constant.py was added to define reusable UI properties such as background and foreground colors. These constants were imported into the main frontend file and applied to the components.

The password field was configured using the show='*' property to hide user input. Buttons such as Refresh , Login , and Create Account were added to the login frame. The Refresh button temporarily displayed a static CAPTCHA value for UI testing.

Basic input validation was implemented using .get() to retrieve user input. If mandatory fields like User ID were left empty, a warning message was displayed. This completed the design and basic validation of the login page interface.

**Observations and Discussions**

It was observed that Tkinter provides an efficient way to design graphical user interfaces and allows easy integration with backend logic in later stages.

## Day 7: Frontend Development – Sign-Up (Create Account) Frame

**Nature of Work**

On the seventh day, a Sign-Up (Create Account) frame was developed in the frontend using Tkinter , similar to the previously created login frame. A separate frame was designed to allow new users to create an account.

The sign-up frame included input fields for:

First Name

Last Name

Phone Number

Email ID

User ID

Password

A function ( def create_account ) was written to handle the sign-up frame logic. Navigation between frames was implemented using button commands. When the Create Account button was clicked on the login page, the application switched to the sign-up frame. A Close button was added in the sign-up frame to return the user back to the login frame.

The frontend was structured to collect user input parameters, which would later be sent to the backend using URL requests and Postman testing.

When the backend and frontend applications were executed through the command prompt, the sign-up interface was displayed successfully. However, on submission, the output showed “Operation Failed” , which was expected at this stage since backend integration for sign-up functionality was not yet implemented.

**Observations and Discussions**

This step helped in understanding frame switching, button commands, and UI flow design in Tkinter before integrating backend services.

## Day 8: Frontend–Backend Integration for Sign-Up and CAPTCHA

**Nature of Work**

On the eighth day, the sign-up functionality developed earlier was connected to the backend services. The output of the previous day showed a “Retry Again” message when the user clicked on the sign-up button, indicating that backend integration was required.

The sign-up logic was updated to check the backend response message. If the response message returned “Operation Successful” , the application displayed “Account Created Successfully” . If the response indicated failure, a warning message such as “Retry with different User ID” or “Correct the entered details” was displayed.

The backend URL for personal data creation was copied and linked to the frontend sign-up function. The application was tested by running both backend and frontend using the command prompt. When valid and unique user details were entered, the account was created successfully. If an already existing User ID was used, the system correctly displayed “Retry with different User ID” .

The created user details were verified in the Firebase Authentication and Realtime Database , confirming successful storage of data.

Additionally, CAPTCHA functionality was integrated into the frontend. A function was created to load the CAPTCHA by fetching values from the backend using a URL request. The Refresh button was linked to this function so that every time it was clicked, a new random CAPTCHA value was generated and displayed in the login frame.

Validation checks were added to ensure that User ID, Password, and CAPTCHA fields were not left empty. If any input field was incomplete, the application displayed the message “Incomplete Details” .

**Observations and Discussions**

This stage helped in understanding frontend–backend communication, response handling, and real-time CAPTCHA generation for improving application security.

## Day 9: Login Integration, Splash Screen, and Dashboard Navigation

**Nature of Work**

On the ninth day, the login functionality was fully integrated with the backend. When the user clicks on the Login button, the entered User ID, Password, and CAPTCHA values are sent to the backend using a request URL. The response message from the backend is captured and displayed on the frontend.

If the entered credentials are correct, the message “Logged in Successfully” is displayed. Upon clicking OK , an empty screen appears, indicating successful login and transition to the next stage.

To improve user experience, a splash screen was created. After successful login, the splash screen displays the message “Application is loading, please wait” . This splash screen is temporarily shown using frame placement and delay logic.

The webbrowser module was imported to enable navigation after login. Initially, a test URL was used to confirm redirection functionality.

For further integration, Sublime Text was used to create an HTML file named dashboard.html . This file was saved inside a folder named templates in the backend. For testing purposes, the dashboard page contained simple content.

In the backend, an API was created to load the dashboard page using render_template('dashboard.html') . The dashboard URL was then linked in the frontend load application function instead of the external test URL.

After running both backend and frontend again, successful login displayed the splash screen, followed by redirection to the Dashboard page , confirming proper integration between frontend, backend, and HTML templates.

**Observations and Discussions**

This phase helped in understanding user flow after login, splash screen implementation, template rendering in Flask, and seamless navigation between frontend and backend components.

## Day 10: Dashboard Enhancement and Update Page Design

**Nature of Work**

On the tenth day, the Dashboard page was enhanced to display personal information such as first name, last name, contact number, and email ID. An Update button was added to allow modification of user details.

The dashboard page was styled using internal CSS . Styling was applied to the header, body, and content sections by defining properties such as height, width, font size, font weight, text alignment, background color, and box shadow. Separate styling was also applied to buttons and paragraph elements, including border radius, button colors, and spacing, using ID selectors ( # ).

After completing the dashboard design, the entire dashboard.html file was copied and saved as a new file named update.html . In the update page, static text was replaced with input fields using input type="text" along with appropriate placeholders for first name, last name, contact number, and email ID. A Submit button was added to send updated values.

In the backend, a new route was created for the update page. Similar to the dashboard route, render_template('update.html') was used to load the update page when requested.

After running the backend server and opening the application in Google Chrome, the dashboard page successfully displayed user details along with the Update button. Clicking on Update loaded the update page , showing input fields with placeholders and a submit option.

**Observations and Discussions**

This step improved understanding of HTML template reuse, internal CSS styling, page navigation, and backend routing for multiple templates in a Flask application.

## Day 11: Active User Handling and Dynamic Data Display

**Nature of Work**

On Day 11, modifications were made to both the backend and frontend code to handle the concept of an Active User after login.

In the backend, logic was updated to store the currently logged-in user ID as an active user . This active user value was then used to retrieve the corresponding personal details from the database. A condition was applied to match the active user ID with the stored User ID in the Firebase real-time database.

Once the correct record was identified, the retrieved values such as first name, last name, contact number, and email ID were passed to the dashboard page using render_template() .

In the frontend (Sublime / HTML files), previously the personal details section showed empty or “unknown” values. This was corrected by using Jinja template placeholders (double curly braces {{ }} ) and matching them with the variable names sent from the backend. As a result, when a user logs in with a valid user ID, their actual data is displayed dynamically on the dashboard page.

Additionally, the Update button on the dashboard page was wrapped inside a <form> tag. Appropriate HTTP methods (GET and POST) were configured so that:

Clicking the Update button redirects the user to the update.html page.

Backend routing was updated to handle these requests properly.

After these changes, when the frontend and backend applications were run:

Logging in with a valid user ID displayed the correct personal details on the dashboard.

Clicking the Update button successfully navigated to the update page for editing user information.

**Observations and Discussions**

This day helped in understanding active session handling, dynamic data binding between backend and frontend, usage of Jinja templates, and navigation between dashboard and update pages using HTTP methods.

## Day 12: Modification of User Details and Update Integration

**Nature of Work**

On Day 12, functionality was implemented to modify and update user personal details in the database.

A backend function was created (for example, modifyUserDetails ) which accepts parameters such as First Name, Last Name, Contact Number, Email ID, and User ID . Inside this function, the database is accessed and a condition is applied to check whether the provided User ID exists .

If the User ID exists, the corresponding personal details are updated in the Firebase real-time database.

If the User ID does not exist, an appropriate message such as “User ID does not exist” is returned.

This API was first tested using Postman by passing valid user details and an existing User ID. On successful execution, the response message “Details updated successfully” was received, confirming that the modification logic was working correctly.

After successful backend testing, the same modify details API was linked to the Submit button in the frontend using a <form> tag and appropriate GET/POST methods .

When the frontend application was executed:

Clicking the Update button redirected the user to the update page.

Entering modified details and clicking Submit updated the data in the database.

Upon returning to the dashboard page and refreshing it, the updated personal details were displayed , confirming end-to-end integration.

**Observations and Discussions**

This activity demonstrated how backend update APIs can be connected to frontend forms, enabling real-time data modification and dynamic display of updated user information.

## Day 13: Task Allocation and Planning

**Nature of Work**

The tasks for enhancing the user interface and dashboard layout were discussed and assigned. Planning was done for improving color consistency and adding additional pages, which were implemented in the subsequent days.

## Day 14: Dashboard UI Enhancement and Layout Design

**Nature of Work**

The dashboard interface was enhanced using CSS Flexbox to align two content sections side by side. The application title was updated to “ML-Based Application” , followed by the project title “To Detect and Predict Genetic Diseases Based on Family History.”

The left panel displays the logged-in user’s personal details, while the right panel presents database-related information along with a Predict button. A logo and a graphical result section were added, including a circular image layout to represent disease risk prediction (high-risk/low-risk).

## Day 15: ML Prediction Page Design and Data Input Interface

**Nature of Work**

A new HTML page named ML.html was created to design the machine learning prediction interface. The project title was displayed as the header. Using CSS Flexbox , the page was divided into two sections.

The left section (70%) contains input fields corresponding to the genetic disease prediction database, including Patient ID, Age, SNP values, and Gene Expression values . Dropdown selection was used for categorical attributes such as Sex and Family History . A Submit (Predict) button was added to send the input data for prediction.

The right section consists of a results container, which is currently empty. Once backend and ML model integration is completed, the predicted disease risk result will be displayed here after submitting the data.

This page is linked to the Predict button on the dashboard, enabling navigation from the dashboard to the ML prediction page. The database structure was reflected through the input labels to ensure consistency between frontend input and backend processing.

---

## Day 16: Log Entry

**Division**

Frontend Development

**Nature of Work**

AIML Training

**Main Activity**

UI enhancement and debugging of Login and Sign-Up pages. Increased frame size, input field size, and button dimensions. Applied consistent color theme and centered headings. Fixed frame resizing issue by updating frame placement inside functions, ensuring full-page layout for both login and signup screens.

## Day 18: Backend Logic Review, Dataset Handling & API Testing

**Division**

AIML Application Development

**Nature of Work**

AIML Training

**Description**

On this day, focus was placed on reviewing backend logic and understanding dataset handling for genetic disease prediction. Backend code execution flow was analyzed using Python-based tools. The structure and purpose of the dataset were reviewed, including genetic markers, family history parameters, and gene expression values.

Basic data preprocessing concepts such as data validation, input consistency, and feature handling were revised. The integration of prediction logic within the backend was studied to understand how disease risk is determined based on user inputs. Flask APIs using GET and POST methods were reviewed to observe how data is transferred between frontend and backend. API responses were tested using Postman to verify correct request–response behavior. Common backend errors and validation issues

were identified and analyzed. This day improved understanding of backend functionality, dataset usage, and API testing.

## Day 19: ML Integration, Prediction Flow & UI Finalization

**Division**

AIML Application Development

**Nature of Work**

AIML Training

**Description**

On this day, machine learning prediction functionality was integrated into the web application for predicting genetic disease based on family history . The Predict button was activated to navigate from the dashboard page to the ML.html page. HTML code was modified using Sublime Text, and backend request handling was updated using POST methods.

The application's color theme was standardized to dark blue across all pages to maintain UI consistency. The flow of user input data from frontend forms to the machine learning model was analyzed. Backend server execution and response handling were reviewed to understand how real-time prediction results are generated and displayed on the dashboard. The prediction output was validated using the dataset to ensure correctness and reliability. This completed the integration and testing phase of the project.

## Day 23: Backend Dataset Preparation & Debugging

**Task Name**

Sample Dataset Creation and Backend Issue Resolution

Record of Main Activities:

On this day, work was carried out on preparing a small sample dataset required for backend processing and testing. A simple Excel dataset was created using binary values (0 and 1) along with categorical output values, and the file was stored in the backend project directory for further use.

The structure of the dataset was analyzed to ensure compatibility with backend logic. Concepts related to accessing datasets within backend applications were studied, along with the use of Python libraries for file handling and Excel file processing. File path handling and dataset accessibility within the backend environment were also reviewed.

In addition to dataset preparation, an issue identified in the update module of the web application was analyzed and resolved. The problem involved incorrect updating of one of the form fields due to a mismatch between frontend form inputs and backend request handling. The issue was fixed by correcting input field mappings and ensuring proper data flow between the frontend and backend. Post-fix testing was performed to confirm that all fields were updating correctly without errors.

This activity helped improve understanding of backend data handling, form validation, and debugging techniques in web-based applications.

**Department**

Artificial Intelligence & Machine Learning

**Nature of Work**

Backend Development and Data Preparation

## Day 27: Dataset Pre-processing and Data Cleaning (Backend)

**Title**

Dataset Cleaning and Validation using Pandas

**Description**

On this day, work was carried out on dataset pre-processing using a sample Excel dataset to understand data quality issues before applying machine learning. The dataset was read using Pandas, and multiple data cleaning operations were performed to ensure data consistency and correctness. Empty cells present in the dataset were identified and removed to avoid invalid inputs during model training. Values with incorrect formats, such as non-numeric entries in numerical columns, were handled by converting them into numeric format where possible and removing invalid rows. Duplicate records were detected and eliminated to maintain dataset integrity. Categorical inconsistencies in the result column were analyzed to identify invalid or inconsistent values. After applying all cleaning steps, a valid dataset was generated, and the total number of rows and columns were computed and displayed. This activity helped in understanding real-world data issues and prepared the dataset for further machine learning operations.

## Day 28: Backend Dataset Handling and Frontend File Selection Design

**Title**

Excel File Reading, Validation Output & Frontend File Upload Interface

**Description**

On this day, backend functionality was extended to read Excel datasets dynamically using a file path parameter. The dataset was successfully loaded and processed using Pandas after validation and cleaning. The cleaned dataset, along with metadata such as total number of rows and columns, was returned as a JSON response and verified using Postman. The backend output was also displayed in the command prompt for verification purposes. A sample dataset was intentionally used to test preprocessing logic and validate the correctness of data cleaning operations before applying them to real project data. In addition to backend processing, frontend design updates were initiated by adding a file selection input using HTML. A file upload interface was created using the <input type="file"> element, allowing users to browse and select datasets from the local system. At this stage, the frontend file selection was implemented without backend integration, serving as preparation for future dataset upload functionality. This day focused on strengthening data handling workflow and improving user interaction design.

## Day 31: Log Sheet

**Department**

Data Science / Full Stack Development

**Nature of Work**

Backend Development & Data Preprocessing

**Task Name**

Backend Function Integration and Output Optimization

**Record of Main Activities of the Day**

The work carried out today focused on improving backend functionality by integrating dataset upload and preprocessing logic into a unified workflow. The previously separate dataset upload function and Excel file reading function were merged to ensure a seamless backend process where a dataset, once uploaded, is immediately preprocessed without requiring additional manual steps.

The backend logic was enhanced to perform data preprocessing operations such as removal of empty values, correction of incorrect data formats, and elimination of duplicate records. This ensured that only a valid and cleaned dataset is considered for further processing.

As part of output optimization, the response mechanism was modified to improve user interface clarity. Instead of displaying the entire dataset on the webpage, the backend was configured to extract and display only essential metadata — specifically, the total number of rows and total number of columns — using a rendered HTML template. This approach improved readability on the frontend while maintaining backend access to the full processed dataset.

The final implementation successfully displays preprocessing confirmation messages along with the total number of rows and columns on the webpage after dataset upload, validating the correct execution of the merged backend logic.

## Day 32 – Work Log

Today we worked on enhancing the dataset properties displayed on the dashboard page. Implemented functionality to display additional dataset details such as column headings below the total number of rows and columns. Modified the backend logic to extract column names dynamically from the uploaded dataset while excluding the target/output column ( Result ) from being displayed.

Additionally, implemented automatic calculation of the uploaded dataset size instead of using hardcoded values. The file size is now computed programmatically during upload and displayed on the dashboard in kilobytes (KB). This involved handling file metadata during the upload process and ensuring the values are passed correctly to the frontend using render templates.

---

## Day 33 – Work Log

Today we enhanced the backend preprocessing logic to support large, real-world datasets. The preprocessing function was modified to remove hardcoded column and row limitations, enabling successful handling of the original dataset containing approximately 4000 rows without causing internal server errors.

Additional updates were made to the upload functionality to preserve user/person details on the dashboard after dataset upload. Previously, these details were getting erased upon uploading a dataset; this issue was resolved by correctly passing and retaining the required data through the render template logic.

Also implemented functionality to ensure that once a dataset is uploaded, its contents are displayed in the backend command line for verification and debugging purposes. This helps confirm that the dataset is being read and processed correctly on the server side.

Overall, the system now supports dynamic datasets, maintains user session information after uploads, and provides backend-level visibility of uploaded data.

---

## Day 35 – Work Log

Today we initiated the literature study phase of the project. Two research papers were studied and documented in detail. For each paper, the following aspects were analyzed: author(s), year of publication, title, problem addressed, dataset used, algorithm employed, and key results.

The first paper focused on machine learning–based prediction of genetic diseases , where the study explored how different ML algorithms can be applied to genetic and clinical data to predict the likelihood of inherited disorders with improved accuracy.

The second paper focused on the use of family history for disease risk prediction , highlighting how familial relationships and historical health data can be leveraged to assess disease risk and enhance early diagnosis and preventive healthcare strategies.

This literature review helped in understanding existing approaches, datasets, and algorithms relevant to genetic disease prediction, and provided a strong theoretical foundation for further system design and implementation.

## Day 36 – Work Log

Today, we continued the literature study phase of the project. The third research paper was reviewed in detail, focusing on SNP-based and gene-expression-based prediction models . The study analyzed how single nucleotide polymorphisms (SNPs) and gene expression data are utilized in predictive models to identify genetic disease risks using machine learning techniques.

For this paper, key aspects such as author(s), year of publication, title, problem addressed, dataset used, algorithms applied, and major results were studied and documented. This paper provided deeper insights into advanced genetic data representations and their effectiveness in improving disease prediction accuracy.

In addition to the literature study, today we also created a soft copy of all project log sheets . The complete set of daily logs has been organized and saved on the laptop, ensuring proper documentation and backup of the project progress.

## Day 37 – Work Log

Today’s work focused on improving the user interface and enhancing the visualization components of the ML prediction module.

The following updates were completed:

Removed the “Result” and “Algorithm” text sections from the dashboard page to simplify the interface and improve clarity.

Adjusted and aligned the input fields and labels in the ml.html page to ensure a cleaner and more professional layout.

Added additional frontend code to integrate a doughnut chart visualization in the result section of the ml.html page.

The doughnut chart is designed to dynamically display prediction output once the model is executed upon clicking the Submit button.

The chart uses a visual risk-based color representation:

Green indicates Low Risk of genetic disease.

Red indicates High Risk of genetic disease.

This enhancement improves user interpretability by converting raw prediction output into a visual representation, making the results easier to understand.

## Day 38 – Work Log

Today's work focused on strengthening the data preprocessing pipeline and preparing the backend structure for model integration. The preprocessing function was enhanced to handle multiple critical steps required for machine learning readiness. Outlier detection and handling were incorporated to ensure extreme values do not negatively affect model performance. Data normalization was implemented to scale numerical features appropriately, and categorical variable encoding was added to convert non-numeric data into a model-compatible format. Additionally, the target column was separated and stored in a distinct variable so that it can be used later for model training and prediction, while ensuring it is not displayed alongside the feature columns and missing value information on the dashboard.

Alongside preprocessing improvements, a new backend route named applyml was introduced in the Spyder backend environment. This route is intended to contain the final machine learning implementation code in later stages. For now, it has been structured to return a simple confirmation response ("okay") to verify routing functionality. The route's behavior was successfully tested and validated using Postman to ensure proper request handling and response generation.

## Day 39 – Work Log

Today's work involved improving the integration between the frontend form and backend logic to enhance the efficiency of the Submit button functionality. The form element and corresponding script tag were properly integrated to ensure smoother communication between user input and backend processing.

The doughnut chart visualization is now successfully being displayed on the ml.html page. To achieve this, dummy prediction values were temporarily provided in the backend, which were then passed to the frontend and connected through the JavaScript code written in Sublime. Although the chart is currently static (based on dummy data), the successful data flow confirms that the frontend–backend integration is functioning correctly and is ready for dynamic model output in the next phase.

In addition to development work, the final paper for the literature study was added to the project report. The paper focuses on a Comparative Study of Machine Learning Algorithms in Healthcare . As with the previous studies, the analysis included author(s), year of publication, title, problem addressed, dataset used, algorithms applied, and key results. This completed the planned literature survey section for the project.

## Day 40 – Work Log

Today's work focused on enhancing the visualization and strengthening the frontend–backend interaction for the prediction module. The previously implemented doughnut chart was replaced with a more intuitive Risk Gauge Meter , designed similar to a speedometer. The gauge visually represents prediction risk levels across three defined zones: Green (0–33%) indicating Low Risk, Yellow (34–66%) indicating Medium Risk, and Red (67–100%) indicating High Risk. A dynamic needle mechanism was implemented to point to the calculated risk percentage, along with a numerical percentage display and a corresponding risk status label for improved interpretability.

Further improvements were made to connect the Submit button directly to backend processing. The form was assigned an identifier ( id="predictionForm" ), and the button was configured to trigger a JavaScript function submitForm() upon submission. Page reload was prevented using event.preventDefault() to ensure smoother asynchronous communication between frontend and backend.

Two primary JavaScript functions were developed. The submitForm() function collects user input data and sends a POST request to the /applyml backend route. The updateGauge() function dynamically

adjusts the gauge needle position, updates the displayed risk percentage, and modifies the status color based on the returned risk value.

On the backend side, the /applyml route was structured to handle prediction requests. At present, it returns dummy JSON data ( {"risk_percentage": 45} ) to validate system flow. This will later be replaced with actual machine learning model predictions.

The complete workflow was successfully verified: when a user clicks Submit, form data is sent to the backend, a risk percentage is returned, and the gauge updates dynamically to reflect the correct risk level (e.g., 45% displaying “Medium Risk” in yellow). This confirms proper frontend–backend integration and dynamic visualization functionality.

## Day 41 – Work Log

Today, we prepared the Feasibility Study section of the project documentation. The feasibility analysis was conducted to evaluate the practicality and viability of the proposed machine learning–based genetic disease risk prediction system.

The study covered the following aspects:

Technical Feasibility:

Analyzed whether the required tools, technologies, and technical expertise are available to successfully develop the system. The project utilizes Python, Flask framework, machine learning libraries, and frontend technologies such as HTML, CSS, and JavaScript, all of which are accessible and manageable within the current technical skill set. The system architecture and implementation requirements were determined to be technically achievable.

Operational Feasibility:

Evaluated whether the system would function effectively in a real-world scenario. The application is designed with a user-friendly interface, clear risk visualization (Risk Gauge Meter), and straightforward input mechanisms. This ensures that users can easily interact with the system and interpret prediction results, making it practically usable.

Economic Feasibility:

Assessed the cost implications of development and deployment. Since the project primarily uses open-source technologies and locally available resources, the development cost is minimal. No significant financial investment is required beyond standard computing infrastructure.

Schedule Feasibility:

Examined whether the project can be completed within the internship duration. Based on the current progress in frontend development, backend integration, preprocessing pipeline, visualization, and literature review, the project timeline is on track and achievable within the allotted internship period.

Overall, the feasibility study confirms that the system is technically viable, operationally practical, economically affordable, and achievable within the defined schedule.

---

## Day 42 – Work Log

Today’s work focused on optimizing the user interface to ensure proper display and alignment of the application on desktop devices. Adjustments were made to the layout structure, spacing, and alignment of various UI components to improve visual consistency and usability. Special attention was given to form elements, dashboard sections, and the Risk Gauge Meter to ensure that all components are properly positioned and clearly visible without overlapping or distortion.

The CSS styling and structural alignment were refined so that input fields, labels, buttons, and result display sections appear organized and professionally formatted when accessed on desktop and laptop

screens. These improvements enhance user experience and ensure that the application maintains a clean and responsive layout in a standard desktop environment.

Overall, today's work strengthened the presentation layer of the application by improving visual structure, readability, and interface consistency.

## Day 43 – Work Log

Today's work focused on further refining the visual design and overall presentation of the application. Enhancements were made to layout alignment, spacing, and color combinations to improve aesthetic consistency and user experience. UI elements were carefully adjusted to maintain uniform margins, balanced spacing between components, and proper structural alignment across different sections of the application.

Color schemes were reviewed and refined to ensure visual clarity, readability, and meaningful representation of system outputs. Particular attention was given to maintaining intuitive color usage in risk visualization components while ensuring the overall interface remains professional and visually appealing.

These refinements contribute to a cleaner, more structured, and user-friendly interface, enhancing both usability and presentation quality of the application.

## Day 44 – Work Log

Today's work focused on improving form validation and reducing incorrect form submissions within the application. Additional validation mechanisms were implemented to ensure that users provide accurate and complete input data before submitting the prediction form. Required fields were properly enforced, and logical input constraints were added to prevent invalid or inconsistent data entries.

Frontend-level validation checks were strengthened to reduce unnecessary backend requests caused by incomplete or incorrect submissions. This helps maintain data integrity, improves system reliability, and enhances overall user experience by guiding users to provide valid inputs before processing.

These improvements ensure smoother operation of the prediction workflow and reduce potential errors during backend processing.

## Day 45 – Work Log

Today's work focused on improving the clarity and usability of the input form by adding proper and descriptive labels for all form fields. Each input element was clearly labeled to ensure users understand the type of data required, reducing confusion and improving overall interaction with the system.

The labeling enhancements contribute to better accessibility, improved user experience, and more structured data collection. This refinement ensures that the prediction module is intuitive, organized, and professionally presented.

With this update, the application interface and workflow have been fully structured, refined, and completed as per the planned scope.

## Day 46 – Work Log

Today's work focused on enhancing user experience and improving interaction feedback within the application. An animated Upload Progress Bar was successfully implemented on the dashboard page to visually indicate dataset upload status. The progress bar appears directly below the "Upload Dataset" button and features a smooth green progress indicator. The animation speed was adjusted for better visual clarity and user perception. The implementation works seamlessly with the existing Flask backend, ensuring synchronized frontend–backend communication during file uploads.

In addition, improvements were made to the prediction result display behavior. Previously, after clicking the Submit button, users had to manually scroll to view the Risk Gauge Meter. This issue was resolved by modifying the logic so that the page now automatically displays the gauge meter upon submission, ensuring immediate visibility of prediction results without requiring manual scrolling. This enhancement significantly improves usability and workflow smoothness.

These updates further refine the application's responsiveness, interactivity, and overall user experience.

## Day 47 – Work Log

Today's work focused on improving input accuracy and enhancing user interaction within the authentication and prediction modules. The input field types were refined by changing appropriate fields to number and text to ensure structured data entry. Validation constraints such as minimum and maximum values were added for SNP count and age fields to prevent unrealistic or invalid inputs. Additionally, the gene expression field was modified to allow decimal values, enabling more precise numerical data entry.

Further enhancements were made to the Login and Signup pages by implementing automatic cursor focus functionality using PyCharm during development. The cursor now automatically appears in the first input field when the page loads, improving usability. Keyboard navigation was optimized so that pressing the Enter key moves the cursor to the next input field seamlessly. Even when navigating back to the page, the cursor remains active and blinking within the form fields, ensuring a smooth and user-friendly login/signup experience.

These refinements improve data validation, usability, and overall interface responsiveness of the application.

## Day 48 – Work Log

Today's work focused on improving application accessibility and efficiency by implementing keyboard shortcut keys for faster navigation across different pages. These shortcuts enhance user experience by allowing quick access to important features without relying solely on mouse interaction. All shortcut integrations were implemented and structured using Sublime.

On the Dashboard page , multiple keyboard combinations were added:

Ctrl + E to open the Update page (Update button functionality).

Ctrl + U to open the dataset upload dialog box (Upload Dataset button functionality).

Ctrl + P to navigate directly to the prediction page (ml.html).

On the Update page , the Escape (Esc) key was configured to return to the previous page, enabling quick navigation without additional clicks.

Similarly, on the ml.html (Prediction) page , the Escape (Esc) key was implemented to navigate back to the previous page. No additional shortcut was required for the Submit button, as pressing the Enter key naturally triggers form submission, maintaining intuitive behavior.

These enhancements significantly improve usability, accessibility, and navigation efficiency, making the application more user-friendly and professionally structured.

## Day 49 – Work Log

Today's work focused on strengthening theoretical understanding of the technologies used in the project. A detailed study was conducted on core backend concepts and supporting Python modules relevant to web application development.

Concepts explored include:

Flask – Understanding its role as a lightweight Python web framework used to build the backend server and manage routing, request handling, and response generation.

Tkinter – Studied as a Python GUI library for desktop applications to understand differences between desktop-based and web-based interfaces.

Pickle – Learned about its role in serializing and deserializing trained machine learning models for storage and reuse.

jsonify – Understood how it converts Python data into JSON format for structured frontend-backend communication.

request – Studied how Flask's request object is used to capture form data and handle HTTP requests.

webbrowser module – Learned how it can be used to automatically open web applications in a browser.

Additionally, core architectural concepts such as the difference between frontend (user interface and client-side interaction) and backend (server-side logic, data processing, and model execution) were reviewed to gain a clearer understanding of full-stack application structure.

This theoretical reinforcement strengthened overall conceptual clarity and understanding of how the complete system operates internally.

## Day 50 – Work Log

Today's work focused on gaining comprehensive theoretical knowledge of Machine Learning algorithms, including both supervised and unsupervised learning techniques.

Under Supervised Learning , algorithms such as:

Linear Regression

Logistic Regression

Decision Trees

Random Forest

Support Vector Machines (SVM)

K-Nearest Neighbors (KNN)

Naïve Bayes

were studied in detail, understanding how they work with labeled datasets for prediction and classification tasks.

Under Unsupervised Learning , algorithms such as:

K-Means Clustering

Hierarchical Clustering

DBSCAN

Principal Component Analysis (PCA)

were explored to understand how patterns and structures can be identified in unlabeled data.

Further analysis was conducted to determine which type of algorithm is most suitable for our project. Since our dataset contains labeled outputs (e.g., disease risk prediction/classification), Supervised Learning algorithms are more appropriate. Among them, classification-based algorithms such as Logistic Regression, Random Forest, or Support Vector Machines are particularly suitable, as the project involves predicting a specific outcome based on input features like SNP values, age, and gene expression.

This study helped in understanding the reasoning behind model selection and ensured that the chosen approach aligns with the nature of the dataset and project objectives.

## Day 51 – Work Log

Today, the dataset structure was finalized for model training. The last column of the dataset was treated as the target variable , while all the remaining columns were considered input features for the prediction model. The previously used hardcoded file path in the readxlfile function was removed to improve flexibility, and OS-based file handling was implemented so that datasets can now be uploaded and accessed from any location in the system. This modification made the dataset loading process more dynamic and adaptable to different datasets.

Further preprocessing steps were applied to prepare the data for machine learning. The processed dataset was divided into training and testing sets , allowing the model to learn patterns from one portion of the data and evaluate its performance on unseen data. Additionally, feature scaling was applied using a scaler to ensure that all numerical values remain within a consistent range, which helps improve model stability and performance during training.

## Day 52 – Work Log

Today's work focused on implementing the machine learning model and integrating it with the backend system . A Logistic Regression model was trained using the processed dataset. Once the model training was completed, the trained model was stored globally within the backend application so that it can be reused whenever predictions are requested by the user.

During the prediction phase, user inputs from the frontend form — including age, gender, family history, SNP values, and gene expression values — are collected and converted into a structured DataFrame with the same feature names that were used during the training process. These inputs are then passed through the same scaler used during training to maintain consistency in data representation.

After scaling, the processed inputs are provided to the trained model, which calculates the probability of disease occurrence . This probability value is converted into a percentage format and returned to the frontend using a JSON response . The frontend then displays this result visually using the dynamic risk gauge meter , indicating low, medium, or high genetic disease risk .

## Day 53 – Work Log

Today's work involved evaluating the performance and limitations of the current prediction model . While the Logistic Regression model was successfully integrated and functioning, it was observed that the algorithm does not effectively capture non-linear relationships present within the dataset. The model appeared to find a strong correlation mainly between the family history feature and the target variable , while other important biological indicators such as SNP values and gene expression data were not contributing as effectively to the prediction process.

In real-world genetic disease prediction, risk is usually determined through complex interactions between multiple biological factors , rather than a simple linear relationship between one feature and the outcome. Because Logistic Regression primarily works well with linear relationships , it may not fully capture these complex patterns present in genetic data.

Therefore, it was concluded that more advanced machine learning algorithms capable of handling non-linear relationships will be explored and implemented in the later stages of the project to improve prediction accuracy and overall model performance.

## Day 54 – Work Log

Today's work focused on improving the prediction performance of the machine learning model used in the project. In the final stage of development, the previously used Logistic Regression model was replaced after evaluating multiple machine learning algorithms to achieve better prediction accuracy. The uploaded dataset first undergoes preprocessing where missing values are removed, duplicate records are eliminated, categorical values are encoded, and numerical data is validated to ensure data quality. After preprocessing, the dataset is separated into feature variables and the target variable , and feature scaling is applied so that all numerical values remain within a consistent range for effective model training.

The processed dataset is then divided into training and testing sets . Multiple machine learning models including Support Vector Machine (SVM), Random Forest, and Gradient Boosting were trained and evaluated to determine which algorithm performs best for the genetic disease prediction task. These models were compared using standard evaluation metrics such as accuracy score, confusion matrix, and classification report to analyze their prediction performance.

## Day 55 – Work Log

Today's work involved selecting and finalizing the most suitable machine learning model for the prediction system. Based on the performance comparison of different algorithms, the Support Vector Machine (SVM) model achieved the highest prediction accuracy and was therefore chosen as the final model for the application. The trained model is stored and later used for prediction when users interact with the system.

When a user enters input values through the frontend prediction form , the data (such as age, gender, family history, SNP values, and gene expression values) is converted into a structured DataFrame with the same feature format used during training. These inputs are then scaled using the same scaler applied during model training and passed to the trained SVM model , which calculates the probability of genetic disease risk.

The model evaluation produced an accuracy score of approximately 0.84 , indicating good predictive capability. The confusion matrix and classification report showed that the model performs very well in identifying the healthy class , achieving high precision and recall for that category. However, it was observed that the model performs slightly weaker in predicting the disease class , meaning that while it detects healthy individuals accurately, predicting disease cases is comparatively more challenging. This analysis helped in understanding the strengths and limitations of the final prediction model.

## Day 56 – Work Log

Today's work focused on further improving the performance and reliability of the final prediction model. Additional enhancements were implemented by addressing the issue of class imbalance in the dataset using balanced class weights , ensuring that the machine learning model pays equal attention to both healthy individuals and disease-risk cases during training. This adjustment helped improve the model's ability to recognize minority class patterns more effectively.

To evaluate the stability and generalization capability of the model, cross-validation was applied across multiple data splits. The results confirmed that the model produced consistent performance with an average accuracy of approximately 83–84% , indicating reliable predictive behavior across different subsets of the dataset.

Further optimization was performed through hyperparameter tuning using GridSearchCV , which systematically tested different parameter combinations to identify the best configuration for the Support Vector Machine (SVM) model. The optimal parameters were determined to be a linear kernel with C = 1 and gamma set to scale . This tuning improved the model's ability to correctly detect disease-risk cases by increasing recall for the positive class while maintaining strong overall accuracy.

After identifying the optimal configuration, the system architecture was simplified to train only the tuned SVM model , reducing computational overhead and improving efficiency while preserving the enhanced predictive performance.

## Day 57 – Work Log

Today's work focused on improving the user interface and informational value of the application to make the system easier to understand for non-technical users. An educational genetics information page was designed and integrated into the project, providing clear explanations of the key biological concepts used in the prediction system. The page explains important topics such as factors affecting genetic diseases, Single Nucleotide Polymorphism (SNP), gene expression, and common genetic risk factors including age, gender, and family history .

Additional explanations were also included to help users understand the input values used in the system , such as the meaning of SNP values (1–10) and gene expression values (1–5) . These descriptions help users interpret the data fields correctly while entering information for prediction. By including this guidance, the system becomes more accessible and user-friendly even for individuals who do not have a technical or bioinformatics background.

To further enhance the scientific credibility of the project, a reference to the National Center for Biotechnology Information was included on the page. This allows users to learn about authentic sources where real genetic datasets and biological research data can be obtained, making the project resemble a practical bioinformatics-based healthcare application .

## Day 58 – Work Log

Today's work involved preparing a PowerPoint presentation to summarize and demonstrate the project developed during the internship, as assigned by the project guide. The presentation was created to clearly explain the overall system and its implementation. It includes important sections such as the tools and technologies used in the project, key features of the application, system modules, and the Python packages imported for backend development .

In addition to the technical components, the presentation also explains the working of the machine learning model , including how the dataset is processed, how the trained model performs predictions, and how the results are displayed through the user interface. To make the presentation more clear and visually understandable, screenshots of the application interface and prediction outputs were added.

These screenshots demonstrate how the system looks during real usage, including the dashboard, dataset upload, prediction form, and the risk gauge result display.

The PowerPoint presentation was completed and finalized as a concise overview of the entire project development process and its final functionality.

## Day 59 – Work Log

Today's work focused on improving the prediction quality and probability variation of the machine learning system by upgrading the model from Support Vector Machine (SVM) to XGBoost , an advanced ensemble learning algorithm based on gradient boosting techniques. The earlier SVM model often produced nearly constant probability outputs, typically around 61%, which limited the variation and interpretability of risk predictions. To overcome this limitation, the model pipeline was updated to use XGBoost, which constructs a sequence of decision trees where each new tree learns from the errors of the previous trees. This approach allows the model to capture more complex and nonlinear relationships between genetic features such as SNP values, gene expression levels, and demographic factors .

During implementation, a backend type-conversion issue was identified and corrected by ensuring that the prediction probability returned by the model was properly converted into a Python float before being sent to the frontend. After resolving this issue, the system successfully generated dynamic probability values , which correctly updated the risk gauge visualization displayed in the user interface.

Further improvements were achieved through hyperparameter tuning , including increasing the number of trees, controlling tree depth, and adjusting the learning rate to optimize model performance. As a result, the final XGBoost model achieved an accuracy of approximately 83% when evaluated on the processed dataset containing around 3,800 samples . This upgrade significantly enhanced both the predictive capability of the model and the clarity of the risk visualization presented to users.

## Day 60 – Work Log

Today's work focused on adding a final usability feature to the system by enabling users to download their prediction results as a PDF report . After the machine learning model generates the disease risk probability, the application now provides a "Download Risk Report" option that allows users to obtain a personalized document summarizing their prediction results.

This functionality was implemented in the Flask backend using the ReportLab library, which dynamically generates a PDF file at the moment the user requests it. The generated report includes important information such as the predicted risk percentage produced by the model, a brief interpretation of the risk level (low, moderate, or high), and the date on which the analysis was performed .

When the user clicks the download button on the results page, a dedicated Flask route processes the request, creates the document in memory, and sends it directly to the browser as a downloadable file. This feature significantly improves the usability of the system by allowing users to save, print, or share their genetic risk assessment , making the application resemble real-world medical decision-support tools that provide printable diagnostic reports.

## Day 61 – Work Log

Today's work focused on correcting minor issues in the application and improving the presentation of generated outputs. Initially, it was observed that the logo image on the dashboard page was not loading properly . After debugging the issue, it was identified that the static folder was not correctly placed inside the backend project structure , and the image path in the HTML code had been hardcoded , which prevented the Flask framework from locating the file. This issue was resolved by restructuring the static directory correctly and replacing the hardcoded path with the url_for() function , which allows Flask to dynamically locate static resources. After implementing this change, the dashboard logo displayed correctly.

Further improvements were made to the PDF risk report generation feature . The report layout was enhanced using adjustments made through Sublime and Spyder , improving its visual presentation. The generated PDF now includes clear borders, structured formatting, and the inclusion of the risk chart along with the predicted values so that users can easily interpret the results. Additionally, a disclaimer statement was added to the report clarifying that the prediction is generated using a machine learning–based system and should be interpreted as a computational assessment rather than a definitive medical diagnosis.

These enhancements improved both the visual quality of the application interface and the professional presentation of the downloadable report .

## Day 62 – Work Log

Today's work involved completing and presenting the PowerPoint presentation of the project , which had been assigned earlier as part of the internship evaluation. The presentation was prepared to clearly explain the development and functionality of the system. It included key sections such as the project overview, objectives of the system, tools and technologies used, imported Python packages, and the different modules implemented in the application .

The presentation also described the system architecture and working flow of the machine learning model , explaining how the dataset is processed, how predictions are generated, and how results are displayed in the user interface. To make the explanation clearer, several screenshots of the application interface and prediction outputs were included to visually demonstrate how the system operates in real time.

During the session, each team member took turns presenting different sections of the project to the project guide. The presentation concluded with a brief summary and conclusion of the project , highlighting the key features and overall functionality of the developed system. The entire project workflow and implementation were explained clearly during the presentation.

## Day 63 – Work Log

Today's work focused on reviewing and consolidating the overall project developed during the internship. A detailed revision of the complete system was carried out, including the backend logic, machine learning model pipeline, and frontend user interface. The workflow of data preprocessing, model training, prediction generation, and result visualization was re-examined to ensure consistency and correctness across all modules.

Additionally, the project documentation, log sheets, and presentation materials were reviewed and organized for final submission. Minor refinements were made wherever necessary to improve clarity, structure, and overall presentation quality.

This review process helped in reinforcing understanding of the system and ensured that the project is complete, well-structured, and ready for evaluation.

## Day 65 – Work Log

Today's work focused on identifying and documenting future enhancements for the project. Based on the current system limitations and user experience considerations, several improvements were planned to make the application more robust, user-friendly, and closer to a real-world medical system.

The following enhancements were noted:

Dataset Accessibility Improvement

Provide users with an option to download a sample dataset , ensuring that the system can be used even by those who do not have their own dataset.

Login Flow Validation Correction

Improve the login system by implementing proper validation checks instead of generic error messages.

UI Enhancement to Resemble a Medical Application

Redesign the interface to make it look more professional and medical-oriented , rather than a basic project interface.

Fill Sample Data Feature

Add a button to automatically fill the form with sample values, helping new users understand input requirements easily.

Detailed Authentication Feedback

Display specific validation messages such as:

CAPTCHA incorrect

User ID not found

Password incorrect

User ID already exists (Signup)

Delete Account Feature

Provide users with the option to delete their account securely , including authentication verification.

Prediction History Feature

Allow users to view their previous predictions in a separate history page for better tracking and usability.

These planned enhancements aim to improve the functionality, usability, and overall quality of the system in future development stages.

## Day 66 – Work Log

Today's work focused on improving the usability of the system by reducing its dependency on user-provided datasets. It was identified that the application is highly dataset-dependent, and users without a properly formatted dataset would not be able to use the system effectively. To address this, a new feature was implemented to guide users and provide them with a ready-to-use dataset.

A new section was added below the Upload Dataset option in the dashboard with the message "Don't have a dataset? Learn more and download our sample dataset here." This link redirects users to a newly created page called dataset information page , developed using Sublime. This page provides detailed guidance on how to use the system with both sample and custom datasets.

The page includes an option to download a pre-formatted sample genetic dataset in Excel format, allowing users to directly test the system without needing to create their own dataset. Additionally,

detailed instructions were provided for users who wish to use their own dataset. This includes clearly specifying the required columns such as age, sex, family history, SNP values (SNP1–SNP10), gene expression values (GeneExp1–GeneExp5), and the output column (disease status) along with their expected data types.

An important disclaimer section was also added, highlighting strict formatting requirements such as matching column names exactly, using lowercase letters, avoiding spaces, and ensuring that the output column is placed at the end of the dataset. A small dataset format example table was included to visually demonstrate the correct structure.

Finally, a “Close Window” option was implemented to allow users to return easily to the dashboard page. This entire feature significantly improves user guidance, reduces errors during dataset upload, and makes the system more accessible to first-time users.

## Day 67 – Work Log

Today’s work focused on improving the login flow and handling user interaction more accurately within the application. Earlier, when the user clicked the Login button, a confirmation message “Logged in successfully” was displayed and the system would automatically redirect to the dashboard page. However, it was observed that even when the user closed the message using the “X” (cancel) option , the application still redirected to the dashboard, which was not the intended behavior.

To resolve this issue, the login logic was modified in the backend using PyCharm to properly handle user responses. A confirmation prompt was implemented in the format “Logged in successfully, proceed to dashboard?” . Based on the user’s action, conditional navigation was introduced. If the user selects “OK” , the system proceeds to the dashboard page. If the user clicks the “X” (cancel) option, the system remains on the same login page without redirecting.

This change ensures that navigation is now controlled based on explicit user intent, improving the correctness of the application flow and enhancing overall user experience.

## Day 68 – Work Log

Today’s work focused on improving the overall visual design and user interface of the application to make it look more professional and structured. The earlier design appeared informal, so the layout, formatting, and color scheme were redesigned to create a more polished and user-friendly appearance. A consistent color palette of dark navy, white, and subtle teal was introduced to give the application a clean and modern look.

Typography was enhanced by selecting Sora for headings and DM Sans for body text , improving readability and visual hierarchy. The layout was reorganized into card-based sections instead of a single long page, making the interface more structured and easier to navigate.

Significant improvements were made to the Dataset Information page , where the column details are now displayed using a two-column grid layout for better clarity. The example dataset table was enhanced with colored badges to represent disease status , and alert messages were styled distinctly to highlight important information. Additionally, subtle glow effects were added to the hero section and download button to improve visual appeal and user engagement.

These changes significantly improved the overall design consistency, readability, and user experience of the application.

## Day 69 – Work Log

Today's work focused on continuing the user interface enhancements across multiple pages to improve usability, structure, and interactivity. On the Dashboard page , a proper two-column grid layout was implemented, with personal information displayed on the left and functional components on the right. The personal details are now presented within a clean card layout, while dataset properties such as rows and columns are displayed using stat cards with prominent numerical values . The Predict option was redesigned as a call-to-action banner , replacing the plain button with a more visually distinct section. Keyboard shortcuts were also displayed using kbd-style hints to improve user awareness and accessibility.

On the ml.html (Prediction) page , major interface improvements were made. SNP input fields were redesigned from plain text boxes into toggle-style selectors (0/1/2) for better usability. Input sections such as SNP markers, personal information, and gene expression values were separated into individual cards for clarity. The risk gauge was redesigned into a donut-style arc with a smooth animated needle , and the result display was enhanced with large, bold percentage values. Additionally, the result panel was fixed on the right side, allowing users to view prediction outcomes without needing to scroll.

The Update page was simplified into a centered card layout , with input fields aligned neatly and labels clearly positioned above each field. A green confirmation box was added to display success messages, along with a subtle keyboard shortcut reminder for improved usability.

On the Genetic Information page , risk factors were presented as separate card elements , and SNP and gene expression values were enhanced with visual severity bars . The section referencing the National Center for Biotechnology Information was redesigned into a clean side-by-side layout with descriptive content and action buttons. A consistent design theme was maintained across all pages using sticky navigation, gradient hero sections, and rounded card components .

These enhancements significantly improved the overall usability, visual consistency, and professional quality of the application.

## Day 70 – Work Log

Today's work focused on maintaining design consistency across the desktop application interface by updating the Login and Signup pages to match the visual theme used in the web application. The color scheme was standardized using navy blue, teal, and white , ensuring a uniform and professional appearance throughout the system.

A central configuration file ( constant.py ) was updated to include reusable design variables such as teal, card background, entry background, and muted tones . This allows all UI components to dynamically reference a single source for styling, making it easier to maintain and update the design across the application.

The Login interface was redesigned using a centered card layout , replacing the earlier scattered positioning of labels and input fields. The layout is now aligned using proper anchoring so that it remains centered regardless of window size. Input fields were enhanced with a dark navy background and teal underline styling , replacing default system inputs for a more modern appearance. The CAPTCHA section and refresh button were also aligned neatly within a structured container for better usability.

Similarly, the Signup page was redesigned to resemble a structured medical registration form , with fields such as first name and last name placed side by side for better organization. A consistent top navigation bar was also added, matching the design of the web pages, ensuring that both desktop and web interfaces follow the same visual identity.

These changes improved the overall design consistency, usability, and professional appearance of the application across platforms.

## Day 71 – Work Log

Today's work focused on improving user guidance and interaction within the prediction form , especially for first-time users who may not be familiar with the required input format. Since the system did not previously provide any default values, a new feature called "Fill Sample Data" button was implemented using JavaScript.

This button is placed above the input form inside a styled dark navy card along with a short description to guide users. When clicked, it automatically fills the form with realistic sample values , including age (45), gender (female), family history (yes), and a meaningful mix of SNP values (0, 1, 2) and gene expression values (decimal values such as 3.4, 5, etc.) . This allows users to quickly understand how the inputs should be entered and test the system without manually filling each field.

To improve user feedback, the button temporarily changes its appearance by turning green and displaying a confirmation state for two seconds , indicating that the sample data has been successfully applied. After this, it returns to its original state. This visual feedback ensures that users clearly understand that the action has been executed.

Additionally, an issue in the Signup page layout was identified and resolved. Earlier, the page did not support proper vertical scrolling due to the use of incorrect layout positioning. This was fixed by implementing a scrollable layout using .pack() instead of .place() , along with defining a proper scroll region. As a result, the Signup page now supports smooth scrolling, ensuring that all input fields are accessible regardless of screen size.

These improvements enhanced both the usability and accessibility of the application, making it more intuitive for new users.

## Day 73 – Work Log

Today's work focused on improving the error handling and user feedback mechanism in both the Login and Signup functionalities. Earlier, the system displayed very generic error messages such as "Retry again" or "User ID or Password incorrect" regardless of the actual issue. This made it difficult for users to understand what exactly went wrong during authentication.

To resolve this, detailed and specific validation checks were implemented in the backend logic using Spyder and PyCharm. In the Login process , the validation flow was restructured to follow a proper sequence. First, the CAPTCHA validation is performed. If incorrect, the system displays "Captcha is incorrect, please refresh and try again." If the CAPTCHA is correct, the system then checks whether the User ID exists in the database. If not found, it displays "User ID not found, please create an account first." Only if the User ID exists, the system proceeds to validate the password , and if incorrect, it shows "Password is incorrect, please try again."

Similarly, improvements were made to the Signup functionality . If a user attempts to register using an already existing User ID stored in Firebase, the system now displays a clear message: "This User ID already exists, please choose a different one." Additional fallback messages such as "Something went wrong, please try again" were also included to handle unexpected issues like network errors.

These enhancements ensure that users receive accurate and meaningful feedback based on their input, improving overall usability, reducing confusion, and making the authentication process more intuitive and user-friendly.

## Day 74 – Work Log

Today's work focused on implementing a secure account deletion feature to give users full control over their data. A new "Delete Account" button was added to the dashboard page, placed below the Update Info option within the personal information section.

When the user clicks this button, a confirmation flow is triggered, prompting the user to enter their password before proceeding. This ensures that only the authorized user can perform this action. If the correct password is entered and the user confirms the action, a success message “Your account has been successfully deleted” is displayed. Upon clicking OK, the user is automatically logged out and removed from the application, preventing further access unless a new account is created.

To implement this functionality, a new Flask backend route was created to handle the account deletion process . This route verifies the user’s credentials by re-authenticating the password and then proceeds to delete the user data from both Firebase Authentication and the Firebase Realtime Database . This ensures that both authentication records and stored personal data are permanently removed.

On the frontend, JavaScript functions were implemented to manage the delete confirmation modal, password input validation, and error handling. If an incorrect password is entered, an appropriate error message is displayed to the user. Additional UI enhancements such as hover effects and warning messages ( “This action is permanent and cannot be undone” ) were included to clearly communicate the seriousness of the action.

Once the account is deleted, the system prevents access using the old credentials, ensuring that the user cannot log in again unless a new account is created. The page is also cleared before closing to prevent navigation back using browser history.

These changes significantly improved data security, user control, and application integrity .

## Day 75 – Work Log

Today’s work focused on code optimization, maintainability, and documentation of the application. It was observed that the frontend code contained repeated styling across multiple HTML pages, making the code lengthy and difficult to manage. To resolve this, a separate common.css file was created, where all shared styles were consolidated (approximately 300 lines). This stylesheet was then linked to all HTML pages using a common reference, significantly reducing redundancy.

As a result of this restructuring, the overall codebase was reduced from approximately 900 lines to around 600 lines , and further simplification was performed by removing redundant inline styles and unused functions. After optimization, no single file exceeded around 400 lines , making the code more readable, modular, and easier to maintain.

In addition to code improvements, manual documentation work was initiated. A detailed explanation of the project code structure was written in a notebook, including descriptions of styling concepts such as CSS root variables, color schemes, and reusable design tokens . Explanations for different modules such as the ML dashboard, update page, and genetic information page were also included. Around 15 pages of documentation were completed, with plans to extend it further.

A minor issue in the Login functionality was also identified and resolved. If the user remained idle on the login page for a long time, attempting to log in would result in a generic retry error due to a network timeout or stale session. This was fixed by wrapping the request in a try-except block with a timeout mechanism , and by automatically refreshing the CAPTCHA after a failed login attempt. This ensured that users do not get stuck with outdated CAPTCHA values and improved overall reliability of the login process.

These changes enhanced the efficiency, maintainability, and robustness of the application.

## Day 77 – Work Log

Today's work focused on adding a new user-specific prediction history feature to enhance the functionality and usability of the application. A new button called "View Your History" was added below the Run Prediction Analysis section on the ml.html page .

When the user clicks this button, they are redirected to a newly created history page , where all previously generated predictions are displayed. This feature is designed to be user-specific , meaning that each logged-in user can only view their own prediction records, ensuring data privacy and separation between users.

An additional name input field was introduced in the prediction form. While this field is not used for model training, it allows users to label their predictions for easier identification. When a prediction is performed, the system now stores relevant details such as user name, age, gender, family history, risk percentage, risk level, and the date of prediction .

A new frontend page called history.html was created using Sublime, where the stored data is displayed in a structured table format . This allows users to view how many predictions they have made and review their past results in an organized manner.

On the backend, additional logic was implemented in Spyder to handle saving and retrieving prediction data , ensuring that each record is associated with the correct user ID. Minor updates were also made to the ml.html page to integrate the new button and connect it with the backend functionality.

This feature significantly enhances the application by enabling data tracking, user personalization, and result history management , making the system more practical and user-centric.

## Day 78 – Work Log

Today's work focused on performing final system testing and validation of all modules developed throughout the project. A structured testing report was prepared to ensure that each functionality of the application is working correctly and producing the expected output.

Testing began with the Login module , where different scenarios were verified, including incorrect CAPTCHA, invalid User ID, and wrong password, ensuring that appropriate and specific error messages are displayed. Similarly, the Signup module was tested to confirm successful account creation and proper handling of duplicate User IDs with accurate feedback messages.

Further testing was conducted on profile management features , including updating personal details and verifying whether changes are reflected correctly. The account deletion functionality was also tested to ensure secure deletion of user data and proper restriction of access after deletion.

The dataset module was tested by uploading both custom datasets and the provided sample dataset, ensuring correct preprocessing and compatibility. The dataset information page was verified for proper guidance and download functionality. The machine learning prediction module was tested to confirm that predictions are generated correctly using both sample and user-entered data.

The prediction history feature was also validated by checking whether multiple predictions are stored correctly and displayed in the history page in a structured format. The PDF report generation feature was tested to ensure that downloadable reports are created accurately with correct data representation.

Additional testing included verifying the genetic information page , ensuring proper content display, and checking overall system behavior under network-related issues , such as timeouts and error handling messages.

Finally, the machine learning model performance was reviewed by analyzing metrics such as accuracy, precision, recall, and classification report , confirming that the model performs reliably with an average accuracy of approximately 83% .

These comprehensive tests ensured that the entire system is functionally correct, stable, and ready for final use or evaluation .

## Day 79 – Work Log

Today’s work focused on the final deployment of the project , marking the completion of the entire application development process. All project files were first reviewed, organized, and consolidated to ensure that the codebase was clean and ready for deployment.

The project was then uploaded to GitHub using standard Git commands such as initialization, staging, committing, and pushing the code to a remote repository. This ensured proper version control and online accessibility of the project.

To make the application compatible with deployment platforms, necessary modifications were made in the backend code. File handling logic was updated by replacing local directory paths with temporary file handling ( /tmp ) , as deployment environments do not support fixed local storage paths.

Deployment was carried out using Render . The platform was connected to the GitHub repository, and the project was prepared for deployment. A requirements.txt file was created to list all necessary Python dependencies, and a Procfile was added to specify the application entry point ( backend_application.py ).

The Flask application configuration was adjusted to ensure compatibility with the deployment environment. A new web service was created on Render, and the project was deployed successfully. Within a short time, the platform generated a live deployment link , making the application accessible online.

This step marked the successful transition of the project from a local development environment to a live, accessible web application , completing the full development lifecycle.

## Day 80 – Work Log

Today marked the final day of the internship , bringing the entire project journey to a successful conclusion. As all technical work had already been completed, the day was dedicated to final formalities and documentation.

We organized and finalized all required materials for submission, including reports and supporting documents. Photographs were taken as part of the documentation process for the final report. We also expressed our gratitude to our internship mentor for his guidance and support throughout the duration of the project.

Certificates were distributed, confirming the successful completion of the internship. Group photographs were taken with the certificates, which were also documented for official records as part of the Batch of 2026 internship completion .

The internship concluded with all deliverables completed, including the project, documentation, presentation, and evaluation. Overall, it was a valuable learning experience that provided practical exposure to full-stack development, machine learning integration, and real-world application deployment .